

Embryonics Fault Tolerant Hardware and Cyclic Redundancy Codes

Praveen Kumar R and Vijay Raj Kumar M

¹ First Author Department of Computer Science and Engineering, PSG Institute of Technology and Applied Research, Coimbatore, India

`praveenrajesh1203@gmail.com`

² Second Author Department of Computer Science and Engineering, PSG Institute of Technology and Applied Research, Coimbatore, India

`21z154@psgitech.ac.in`

Abstract. Embryonics is a emerging and advance research field. This uses embryonic development from biology and is used in perspective of the hardware circuit. Our proposed worked is to produce self-correcting and self fault tolerant Circuit. In this, embryonics technique is used to do so. Here, the cyclic redundancy codes are used for implementation of the fault-tolerant adder.

1 Introduction

1.1 Introduction

In this embryonic based circuit, biologically if one cell is problem, it is replaced by another when a fault is detected. Similarly, in circuits one fault component is replaced with another on it own once problem detected. The adders are arranged in a matrix i.e grid pattern arrangement based on the number of bits used. The individual adders are connected with MUX circuit which acts as valve for the adder and this is controlled by the Circuit controller based error detected when compared to the truth table for the inputs.

The Cyclic Redundancy Codes (CRC) are error-detection codes which are used in networking and data storage to detect the error occurring in the data bits. Here, the polynomial multiplication is used for encoding and polynomial division for decoding. The adder circuit is implemented in the embryonic structure and the adder is compared with the truth table/golden reference. This truth table is made as a fault-tolerant in which invalid bit can be detected and corrected respectively.

1.2 Abbreviations and Acronyms

Var	Name
$G(X)$	Generated Polynomial
$A(X)$	Encoded Value
$E(X)$	Encoded Polynomial
A, B	Inputs
C_i	Carry In
C_o	Carry Out

2 Literature Works

Researchers have viewed the domain of Evolvable Hardware - EHW from two different, yet closely connected, viewpoints [1]. The first is the use of Evolutionary Algorithms - EAs for circuit synthesis and design as an alternative to conventional methods utilized, which is referred as Evolutionary Electronics. The second involves using EAs to create a self-configurable, evolvable, which is capable of adapting its structure to achieve desired behaviour. The Evolvable Hardware as well as the Embryonic Hardware aim to build circuits with some self-healing characteristics and draw inspiration from biological concepts. But differ significantly in many areas, including design purpose, fault-tolerant concept, reconstruction technique, and granularity. Fault tolerance and Fault prevention are the two main factors which maximizes the reliability of Embryonics. A specially designed Genetic Algorithm [2] and Built In Self Test (BIST) are employed in the architecture of embryonics to enhance error correction ability. The configuration data of each embryonic cell is decoded using 'Cartesian Genetic Programming' (CGP) [2]. The CGP algorithm offers also the detection at gate level. An effective centralized BIST can coordinate with the self-repair mechanisms in the individual cells to initiate repair actions for any detected faults. An additional mode referred as 'Self-Test Mode' of the BIST is employed which when activated, conducts various tests to evaluate the fabric's integrity. A multi Look Up Table (LUT) based approach [3] has been developed for making versatile embryonic cells based on various applications. Error detection and correction mechanisms are provided at both the configuration memory and array. Extended hamming code capable of detecting both 1 and 2 bit errors and correcting single bit error is used. In array level, Column elimination which is a reconfiguration mechanism is used. However resource utilization ratio becomes a major concern in this methodology.

Also cloning [4] is applied to the fundamental embryonic architecture to extend to higher applications. The work [4] also focusses on extension of linear embryonics structure to 2 Dimension. Cloning in two directions, horizontal and vertical, is introduced. This structure is applied to implement a 4x4 multiplier, where 16 cells are needed to perform the multiplication. Apart from the combinational applications such as adder, multiplier, Embryonics can also be extended to perform sequential applications such as Synchronous counters [5]. Flip-flops and

a separate clock for the logic block must be added to the cell's design in order to implement sequential logic in the embryonic cell. The fundamental distinction is the addition of a separate clock signal for the sequential section of the embryonic cell. The notion proposes a "sequential embryonic fabric" made up of two cells. The quantity of flip-flops needed depends on the particular demands for parallel and serial outputs. No matter what this number is, the cell design is constant, and cloning is employed to load data consistently throughout all cells. [5]. But the challenge of Failure Detection with High Coverage [6], which involves improving the fault detection coverage without increasing hardware overhead still persists in the above methodologies.

Regarding the fault tolerant component, A technique for handling the transient fault cells [7] is presented by the Fault-Cell Reutilization Self-healing Strategy (FCRSS). This approach combines two approaches - reconfiguration and removal procedures. Cells with temporary defects in the embryonic circuit array could be utilized at the same time to take up the functions of the cells on their left side in the same row as part of the fault-cell elimination process. In order to achieve fault-cell reutilization, transitory fault-cells in a transparent state can be modified. For fault predictions contemporary Deep learning-based formulations [8] are also put forth. The accuracy and latency of developing hardware are then evaluated using the proposed fault prediction methods and the proposed self-healing technology. These techniques use Principal Component Analysis (PCA), Robust Principal Component Analysis (RPCA), Multilayer Perceptron (MLP), Exponential Linear Unit Long Short-Term Memory (ELSTM), and Fast Fourier Transform (FFT) in combination. But the challenge of designing novel cell structure [6] which requires less latency and overhead becomes a matter of concern in the above processes. [9] [10] [11] [6] [12] [4] [5] [8]

3 Background Work

3.1 Unicellular Self Check Embryonic Cell Design For 8-Bit Adder

Embryonic cells, which mimic the behaviour of biological embryonic cells are structured as a two-dimensional array, consists of both functional and spare cells for fault recovery. Each cell contains various modules as follows address generator, gene, function, control, input/output, and self-repairing units. To address this, a new approach involves a shared self-repair unit, configuration memory, and control unit, optimizing the architecture for an 8-bit adder circuit. This adder circuit is divided into sum and carry parts, designed using 4:1 and 8:1 multiplexer (MUXes) for input selection. The MUXes utilize 26 selection bits to choose inputs from input1 and input2 registers for each bit addition, with their positions affecting selection. Carry MUXes contain two bits, "0" and "1," stored alongside the configuration data. The entire operation is performed within a single embryonics adder cell, which then passes the sum and carry outputs to a self-checking unit through the control module which on detecting error transfers the control to the spare cell else to the next cell.

3.2 Self Check Unit

3.3 Drawbacks In Error Detection Method

One of the primary drawbacks of comparison-based techniques is their requirement to triple the logic circuits, which significantly increases hardware complexity and resource utilization. Also the power consumption of comparison-based techniques is considerably higher, as maintaining multiple copies of circuits continuously consumes energy. Scalability is another major issue with comparison-based approaches; as the system complexity or data size increases, the need to triple the hardware resources escalates rapidly, leading to exponential growth in cost. Further, comparison-based techniques offer less efficiencies with transient or soft errors caused by factors like radiation or electromagnetic interference. These errors are often temporary and require dynamic handling, which static, hardware-based approaches like TMR are not specifically designed to manage.

3.4 Information Redundancy

Information Redundancy is one of the main ways to apply fault-tolerance. Techniques for information redundancy rely on encoding algorithms that add redundant data to the message. By using these techniques, it is ensured that in the unlikely event that some data is corrupted, the redundancy bits will be able to identify and perhaps fix the faults without requiring the data to be retransmitted. Information redundancy is primarily used to increase a system's fault tolerance by allowing it to recognize and manage data discrepancies brought on by noise, signal deterioration, or hardware malfunctions.

3.4.1 Encoding

Encoding involves computing a codeword for a given data. It uses the code's rules to transform a binary k -tuple that represents the data into a codeword with length n . The number of check bits needed to complete the encoding is given by the difference $(n - k)$ between the length of the data (k) and the codeword (n).

3.4.2 Decoding

Decoding refers to determining a codeword for a given set of data. After reading a codeword, a decoder applies the code's rules to try and reconstruct the original data. For instance, in the event of no error, the codeword for a separable code is simply truncated by the number of check bits by the decoder. It is necessary to fix any errors before the data may be retrieved.

3.4.3 Information Rate

To encode binary k -bit data, we need a code C consisting of at least 2^k codewords, since any data word should be assigned its own codeword from. On the other hand, data of length $k \leq \lceil \log_2 C \rceil$ bits is encoded using a code of size C . The information rate of the code is defined as the ratio k/n . The redundancy of the code is determined by the information rate.

3.5 Cyclic Codes

Cyclic codes, subclass of linear codes have an additional structure due to their cyclical nature, which makes encoding and decoding them with linear feedback shift registers (LFSRs) simple. A common form of error-detecting code in digital networks, data storage, and communication systems is Cyclic Redundancy Code (CRC). CRC is an efficient and simple algorithm that detects errors in transmitted or stored data by appending a checksum to the data.

3.5.1 Cyclic Encoding

For encoding data in a cyclic code, the polynomial representing the data is multiplied by a polynomial called the generator polynomial. The multiplication of the data polynomial $d(x) = d_0 \oplus d_1x \oplus d_2x^2 \oplus \dots \oplus d_{k-1}x^{k-1}$ by the generator polynomial $g(x) = g_0 \oplus g_1x \oplus g_2x^2 \oplus \dots \oplus g_rx^r$, where $r = n - k$, can be implemented using an LFSR. ($\oplus$ denotes modulo 2 adders). The multiplication is carried out as follows. Initially, all register cells are set to 0. The coefficients of the data and codeword polynomial appear on the input and output lines, with the high-order coefficient first. As the first data coefficient d_{k-1} appears at the input, the output becomes $d_{k-1}g_r$, which is the correct coefficient c_{n-1} of the codeword polynomial. After the shift, the register cells are loaded with $d_{k-1}g_0, d_{k-1}g_1, \dots, d_{k-1}g_{r-1}$, the input becomes d_{k-2} , and the output becomes $d_{k-1}g_{r-1} \oplus d_{k-2}g_r$, which is the correct coefficient c_{n-2} . At the next clock cycle, the content of the register cells becomes $d_{k-2}g_0, d_{k-1}g_0 \oplus d_{k-2}g_1, d_{k-1}g_1 \oplus d_{k-2}g_2, \dots, d_{k-1}g_{r-2} \oplus d_{k-2}g_{r-1}$, the input becomes d_{k-3} , and the output becomes $d_{k-1}g_{r-2} \oplus d_{k-2}g_{r-1} \oplus d_{k-3}g_r$, which is the correct coefficient c_{n-3} . The multiplication process continues in a similar way.

The degree of the generator polynomial determines the error-correcting capabilities of the cyclic code. An (n, k) cyclic code can detect burst errors affecting $n - k$ bits or less. Encoding can be performed with the help of an LFSR (Linear Feedback Shift Register). Initially, all registers are fed with 0. The next state equations can be derived as follows:

$$\begin{aligned} s_0^+ &= d(x); \\ s_1^+ &= s_0; \\ s_2^+ &= s_1 \oplus d(x); \\ c(x) &= s_2 \oplus d(x) \end{aligned}$$

3.5.2 Cyclic Decoding

Decoding is performed by dividing the codeword by the generator polynomial available. The division of the codeword polynomial

$$c(x) = c_0 \oplus c_1x \oplus c_2x^2 \oplus \dots \oplus c_{n-1}x^{n-1}$$

by the generator polynomial

$$g(x) = g_0 \oplus g_1x \oplus g_2x^2 \oplus \cdots \oplus g_rx^r,$$

where $r = n - k$, can be implemented by an LFSR. The next-state equations for this LFSR are given by:

$$s_0^+ = s_2 \oplus c(x)$$

$$s_1^+ = s_0 \oplus s_2$$

$$s_2 = s_1$$

$$d(x) = s_2$$

Initially, all register cells are set to 0. The coefficients of the codeword and data polynomials appear on the input and output lines, with the high-order coefficient first. It takes $r - 1$ clock cycles for the first input coefficient to reach the last register cell of the LFSR. Therefore, the output is 0 for the first $r - 1$ clock cycles. Afterward, $c_{n-1}g_r$ appears at the output. It corresponds to the coefficient d_{k-1} of the data polynomial. For each nonzero quotient coefficient d_i , where $i \in \{0, 1, \dots, k - 1\}$, the polynomial $d_i g(x)$ is subtracted from the dividend. The feedback connections accomplish this subtraction. After a total of n clock cycles, all coefficients of the data polynomial are computed.

4 Proposed Work

4.1 Adder Check

The adders are connected in a grid matrix manner and each adder is checked with the truth table stored whenever the adder is implemented. In this the adder inputs A, B, C_i and the adder output values sum and C_o is taken and is encoded using the generating polynomial $G(x)$. Here, $G(x)$ is $X^3 + X + 1$ which is one of the primitive polynomial and is encoded using the polynomial multiplication. This encoded value $A(x)$ is checked with the stored truth table existing encoded polynomial $E(x)$. These polynomials are represented in terms of 1's and 0's as a bit string with 1's in the place of the respective polynomial terms. Here, the encoded polynomial is of 8 bits in length.

For example: if the polynomial is $X^6 + X^3 + 1$, then the bit string could be as 01001001 (i.e. $X^7X^6X^5X^4X^3X^2X^1X^0$)

If the calculated encoded bit string $A(x)$ is equal to the respective polynomial bit string $E(x)$ present in the truth table, then the adder has no error. If it is not equal then the adder has some fault and the adder is removed from the embryonic connection.

```
if(A(x)==E(x)) then NO ERROR
else ERROR
```

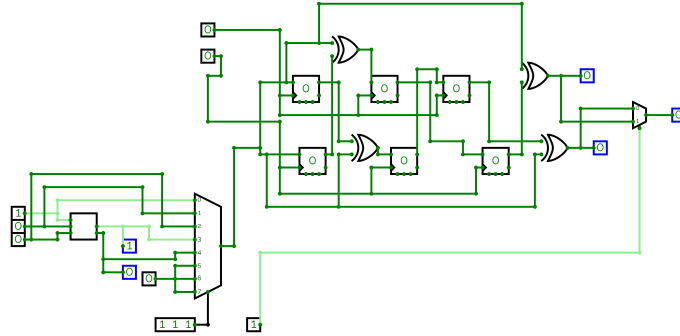


Fig. 1. Polynomial Multiplication.

4.2 Truth Table Check

The truth table contains the adder inputs bits, adder input-output bits and their respective encoded polynomial bits. The adder correctness is checked with this table's encoded polynomial. The truth table or golden reference can also get error or anomalies. Hence, this table should be fault-tolerant. This is done by using the same Cyclic Redundancy Codes method. In this the polynomial stored is in encoded form, so we can use polynomial division check the correctness of the truth table. While on division, if the resultant in the same respective 5 bits as in the truth table i.e full adder value $(A, B, C_i, \text{Sum}, C_o)$, then the table has no error. If the resultant is other then the respective 5 bits, then the truth table has error and it needs to be corrected. This is because the encoded polynomial is completely divisible by the same generated polynomial $G(x)$ when there is no error or bit change. If the bits are changed, then there the remainder is generated along with the answer.

If error is occurred in the truth table, then the respective data bits are used to rectify the encoded polynomial bits in the table. This is by encoding the data bits $(A, B, C_i, \text{Sum}, C_o)$ with the help of the generating polynomial $G(x)$ and then this is compared with the polynomial bits in the table. By this the exact location can be found and the respective bits can be flipped to correct the truth table.

If $(E(x) / G(x) == \text{remainder})$ then ERROR
 Else NO ERROR

5 Implementation

5.1 Adder Check

Let us take an example let the adder inputs (A, B, C_i) be 010. Then the adder produces the output as (sum, C_o) , for this output is 10. This 5 data bits are

DATA BITS	FULL ADDER VALUE	ENCODED POLYNOMIAL
000	00000	00000000
001	00110	00111010
010	01010	01001110
011	01101	01111111
100	10010	10100110
101	10101	10010111
110	11001	11100011
111	11111	11011001

Fig. 2. Truth Table.

encoded using the generated polynomial $G(x) = X^3 + X + 1$. In this the 5 bit data string 01010 are converted into polynomial representation as $X^3 + X$ based on their 1's position. This encoded polynomial bits is checked with the polynomial bits in the truth table for the respective data 010. If the both values are same ,then there is no error in the adder.Else error is present in the adder and it must be replaced.

5.2 Truth Table Check

The truth table is checked frequently once in a while to ensure correctness of the truth table. Initially the truth table has the data bits and its respective encoded polynomial bit string. This polynomial bit string should be kept error-free ,since this is required for the adder correction check. Sometime the bits in the polynomial bit string which is stored may change and leads to false/incorrect value. So this table should be kept error-free. Let us consider an encoded polynomial $E(x)$, 01001110 for 010 data bits. This represented as $X^6 + X^3 + X^2 + X$ based on their 1's position. This is divided by the generated polynomial $G(x) = X^3 + X + 1$ using polynomial division/decoded. If the resultant is 01010 which is the respective 5 bit data string for the polynomial then there exist no remainder and the encoded polynomial in the truth table is error-free.

If the encoded polynomial $E(x)$ is 01101110 and resultant is other then the 5 bit data string as 01110, then there exist a remainder and an error (i.e) bit change in happened in the encoded polynomial $E(x)$ of the truth table. Then the 5 bit data string 01010 is encoded with the generated polynomial $G(x)$, this produces a polynomial bit string as 01001110 which is a correct encoded polynomial $E(x)$. This is stored instead of the error encoded polynomial bits in the truth table or the bits in the error polynomial bits can flipped to correct the table.In this on comparing the new encoded polynomial and the error polynomial we get a remainder as 00100000 (i.e) the error occurs at X^5 bit and this can be flipped.

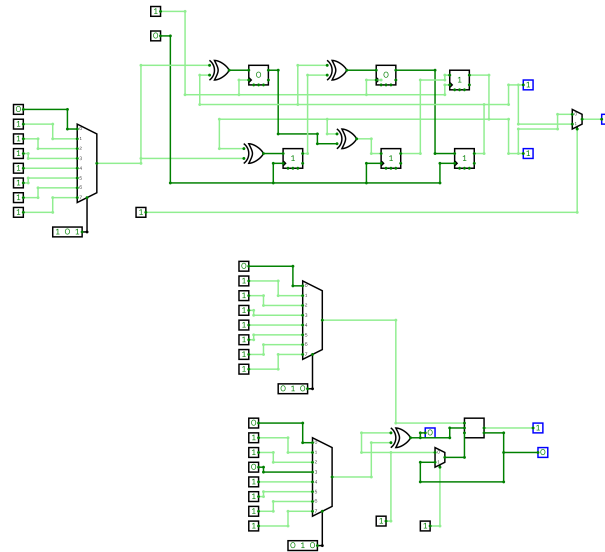


Fig. 3. Polynomial Division.

The above circuit is polynomial division and the bottom circuit is for comparison of new polynomial and the error polynomial to get the remainder.

6 Analysis

6.1 Adder Check

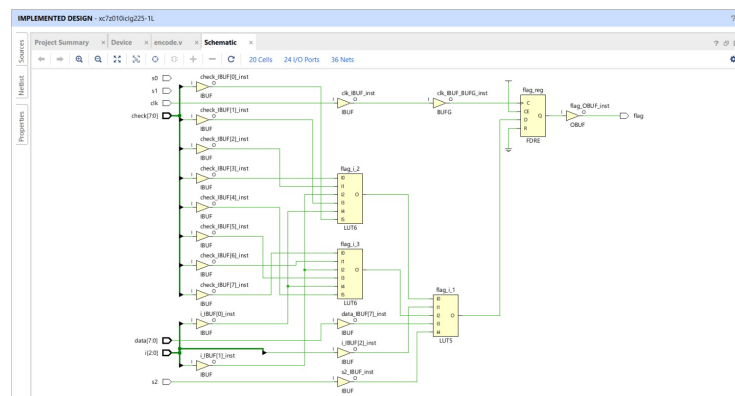


Fig. 4. Adder Check Circuit.

The above circuit is for adder result checking.

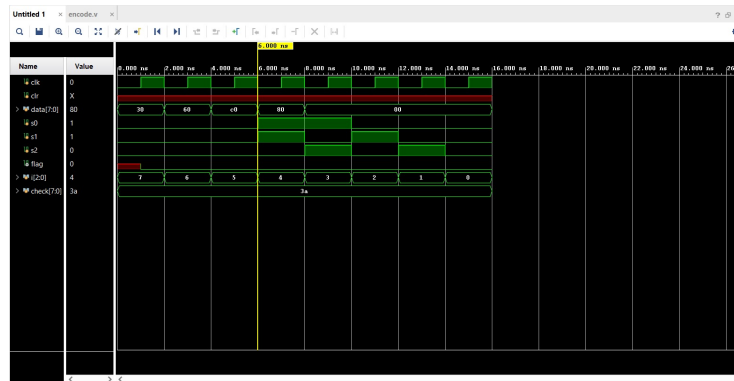


Fig. 5. Adder Circuit Analysis.
The above is the analysis for the adder check.

6.2 Fault Tolerant

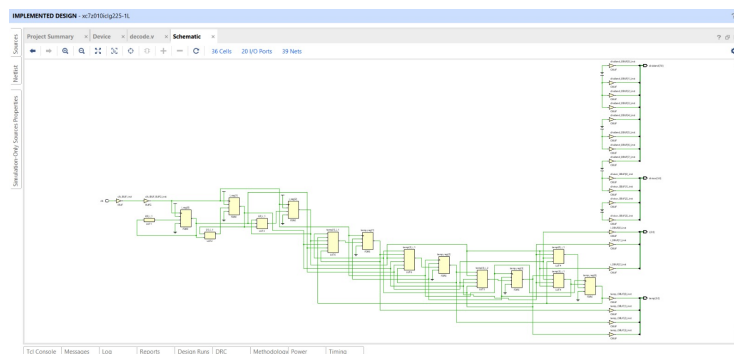


Fig. 6. Adder Check Circuit.
The above circuit is for fault tolerant checking.

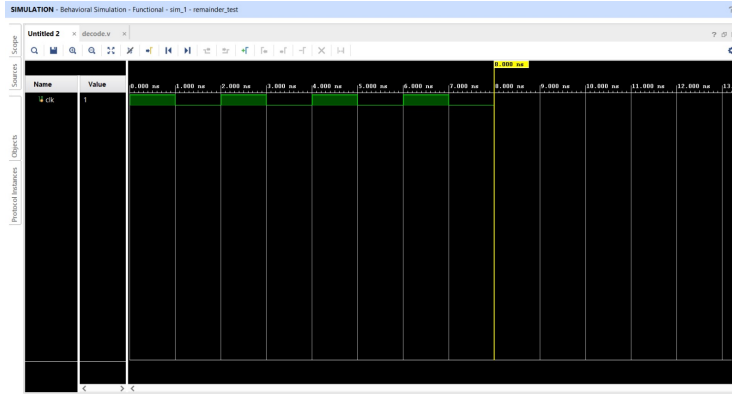


Fig. 7. Fault Tolerant Circuit Analysis.
The above is the analysis for the Fault Tolerant Circuit.

7 Conclusion

This Cyclic Redundancy Codes is one of the methods for error-detection methods. In this, polynomial multiplication and division is used for encoding and decoding of data bits. The adder and truth table can be maintained error-free by this.

References

1. Elhadj Benkhelifa, Anthony Pipe, and Ashutosh Tiwari. Evolvable embryonics: 2-in-1 approach to self-healing systems. *Procedia CIRP*, 11:394–399, 2013. 2nd International Through-life Engineering Services Conference.
2. Gayatri Malhotra, Punithavathi Duraiswamy, and J. K. Kishore. Ga evolved configuration data for embryonic architecture with built-in self-test. In Ajith Abraham, Sabri Pllana, Gabriella Casalino, Kun Ma, and Anu Bajaj, editors, *Intelligent Systems Design and Applications*, pages 34–43, Cham, 2023. Springer Nature Switzerland.
3. Yuan Zhang, Youren Wang, Shanshan Yang, and Min Xie. Design of a cell in embryonic systems with improved efficiency and fault-tolerance. In Lishan Kang, Yong Liu, and Sanyou Zeng, editors, *Evolvable Systems: From Biology to Hardware*, pages 129–139, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.
4. Gayatri Malhotra, Joachim Becker, and Maurits Ortmanns. Novel field programmable embryonic cell for adder and multiplier. In *Proceedings of the 2013 9th Conference on Ph.D. Research in Microelectronics and Electronics (PRIME)*, pages 153–156, June 2013.
5. Gayatri Malhotra, A. M. Nagalakshmi, S. Sudhakar, and Subramanya Udupa. An architecture of sequential embryonic cell for counter design. In *2014 International Symposium on Fundamentals of Electrical Engineering (ISFEE)*, pages 1–6, Nov 2014.

6. Zhuo Qingqi, Qian Yanling, Li Yue, Wang Nantian, and Li Tingpeng. Embryonic electronics: State of the art and future perspective. In *2013 IEEE 11th International Conference on Electronic Measurement Instruments*, volume 1, pages 140–146, Aug 2013.
7. Xiaoliang YU Rui YA Yan CH Youren WA Zhang Z, Yao QI. A self-healing strategy with fault-cell reutilization of bio-inspired hardware. *Chinese Journal of Aeronautics*, 2019.
8. Kasem Khalil, Omar Eldash, Ashok Kumar, and Magdy Bayoumi. Intelligent fault-prediction assisted self-healing for embryonic hardware. *IEEE Transactions on Biomedical Circuits and Systems*, 14(4):852–866, Aug 2020.
9. Andy Tyrrell, Eduardo Sanchez, Dario Floreano, Gianluca Tempesti, Daniel Mange, J. Moreno, Jay Rosenberg, and Alessandro Villa. Poetic tissue: An integrated architecture for bio-inspired hardware. volume 2606, pages 129–140, 03 2003.
10. R Canham and A Tyrrell. An embryonic array with improved efficiency and fault tolerance. In J Lohn, R Zebulum, J Steincamp, D Keymeulen, A Stolica, and MI Ferguson, editors, *2003 NASA/DOD CONFERENCE ON EVOLVABLE HARDWARE*, pages 265–272, United States, 2003. IEEE Computer Society. NASA/DoD Conference on Evolvable Hardware ; Conference date: 09-07-2003 Through 11-07-2003.
11. V. Sahni and V.P. Pyara. An embryonic approach to reliable digital instrumentation based on evolvable hardware. *IEEE Transactions on Instrumentation and Measurement*, 52(6):1696–1702, Dec 2003.
12. Wang Nantian, Qian Yanling, Li Yue, Zhuo Qingqi, and Li Tingpeng. Survey on evolvable hardware and embryonic hardware. In *2013 IEEE 11th International Conference on Electronic Measurement Instruments*, volume 2, pages 1021–1026, Aug 2013.